

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF11a:Suchen und Sortieren:

0. Praktischer Vorlauf

ÜBUNG LF11a:subj-01.search-sort:01

- ☐ Teilen Sie sich in zwei Gruppen auf und wählen Sie in jeder Gruppe je eine Sortiererin.
- ☐ Sie erhalten 8 Spielkarten: 7 8 9 10 B D K As. Mischen Sie diesen Stapel und geben Sie den Ihrer Sortiererin.
- ☐ Gruppieren Sie sich in Ihrer Gruppe um sie und beobachten Sie genau, was sie tut, wenn sie die Karten sortiert.
- ☐ Sammeln Sie in Ihrer Gruppe Ihre Beobachtungen und bringen Sie die Schritte in eine Form, dass Sie einer anderen Sortiererin aus der anderen Gruppe Schritt für Schritt dazu anleiten können, Ihren 'Algorithmus' auszuführen

Lösung:

- Die Schülerinnen werden wahrscheinlich eine Variante des *Insertion Sorts* und/oder des *Selection Sorts* verwenden.
 - Die Lehrerin bringt den Namen des Algorithmus ein und führt den Algorithmus dann in seiner reinen Form vor
 - *Insertion Sort* [→ **ZP:Sheet:5**]
 - *Selection Sort* [→ **ZP:Sheet:8**]
-

ÜBUNG LF11a:subj-01.search-sort:02

- ☐ Versammeln Sie sich um die Lehrerin.
- ☐ Beobachten Sie genau, was sie tut.
- ☐ Lehrerin mischt den 8-Karten-Stack und führt unkommentiert den *Bubble-Sort* vor.

- ☐ Sammeln Sie Ihre Beobachtungen und bringen Sie die Schritte in eine Form, dass das Funktionieren des Bubble-Sorts erklären können

Lösung: Immer wieder paarweiser Kartenswap. [→ **ZP:Sheet:9**]

ÜBUNG LF11a:bj-01.search-sort:03

- ☐ Beantworten Sie die Frage, warum der *Insertion Sort* "Insertion Sort" heißt.
- ☐ Beantworten Sie die Frage, warum der *Selection Sort* "Selection Sort" heißt.
- ☐ Beantworten Sie die Frage, warum der *Bubble Sort* "Bubble Sort" heißt.

Lösung:

- *Insertion Sort* : weil man immer die oberste Karte vom unsortierten Stapel nimmt und an der richtigen Stelle im Sortierstapel **einfügt**.
 - *Selection Sort* : weil aus dem unsortierten Stapel die von der Zielreihenfolge her jeweils nächste Karte **auswählt** und an die feste Stelle im Sortierstapel einfügt.
 - *Bubble Sort* : weil die Karten auf falschen Positionen schrittweise nach oben an die je richtigen wandern
-

ÜBUNG LF11a:bj-01.search-sort:04

- ☐ Betrachten Sie die Grafik zum Quicksort. [→ **ZP:Sheet:10**]
- ☐ Sammeln Sie Ihre Beobachtungen und formulieren Sie die auszuführenden Schritte.
- ☐ Beantworten Sie insbesondere die Frage, wo die Sortierarbeit stattfindet.
- ☐ Klären Sie anhand der Grafik, woraus ein rekursiver Prozess besteht.

Lösung:

- Sortiererin
 - erhält unsortierten Stapel,
 - gibt den, sofern er ≤ 1 Karten enthält, als sortiert zurück,
 - sucht ansonsten ein von der Wertigkeit her ungefähr mittiges Vergleichselement (Pivot-Element)
 - splittet den Stapel in zwei Unterstapel auf: einen mit allen Elementen $<$ Pivot-Element und den anderen mit denen $\geq$ Pivot-Element

- beauftragt andere Sortiererinnen die beiden Unterstapel nach derselben Methode zu sortieren.
 - nimmt die sortierten Unterstapel wieder in Empfang und legt den mit den kleineren Karten auf den mit den größeren
 - gibt den erzeugten Stapel als sortiert zurück.
 - Die Sortierarbeit findet vor der Unterbeauftragung statt (auf dem Rekursionshinweg).
 - Bei einem rekursiven Prozess ruft jeder Schritt sich selbst mit veränderten Input wieder auf.
 - Am Anfang eines jeden Aufrufs wird die Abbruchbedingung getestet.
-

ÜBUNG **LF11a:bj-01.search-sort:05**

- ☐ Betrachten Sie die Grafik zum Mergsort. [**→ ZP:Sheet:11**]
- ☐ Sammeln Sie Ihre Beobachtungen und formulieren Sie die auszuführenden Schritte.
- ☐ Beantworten Sie insbesondere die Frage, wo die Sortierarbeit stattfindet.
- ☐ Beantworten Sie die Frage, warum er Merge-Sort heißt.

Lösung:

- Sortiererin
 - erhält unsortierten Stapel,
 - gibt den, sofern ≤ 1 Karten enthält, als sortiert zurück,
 - teilt die Stapel in zwei von der Anzahl her ungefähr gleich große Stapel
 - beauftragt andere Sortiererinnen die beiden Unterstapel nach derselben Methode zu sortieren.
 - nimmt die sortierten Unterstapel wieder in Empfang
 - sortiert den einen in sich sortierten Stapel in den anderen in sich sortierten ein (mergt die beiden)
 - gibt den erzeugten Stapel als sortiert zurück.
- Die Sortierarbeit findet nach der Unterbeauftragung statt. (Auf dem Rekursionsrückweg)
- Die von den Unteragenten zurückgelieferten Sub-Stapel sind in sich sortiert, müssen aber noch entsprechend gemergt werden.

1. Allgemeines

Grundsätzlich: [**→ ZP:Sheet:2**]

Suche und Sortierung brauchen einander:

- Manche Suchverfahren setzen einen sortierten Suchbereich vor.
- Manche Sortierverfahren müssen
 - das zu einem Platz passende Element aus einer Menge heraussuchen oder
 - den für ein Element aktuell passenden Platz finden.

Zusammenhand: [→ ZP:Sheet:3]

- **Suchen** meint, ein Element in einer Menge (unsortiert) oder Struktur (sortiert) zu finden (und z.B. seine Position als Erfolgskriterium zurückzugeben):
 - **Lineare Suche**: finde das Suchelement in einer **unsortierten Liste**.
 - **Binäre Suche**: finde das Suchelement in einer **sortierten Liste**.
 - **Breitensuche**: finde das Suchelement in einem **gerichteten azyklischen Graph** (z.B. Baum), in dem jeweils - rekursiv - nach der Evaluation eines Vaterknotens erst der Reihe nach all seine Töchter überprüft werden, bevor es mit den Enkelinnenknoten weitergeht.
 - **Tiefensuche**: finde das Suchelement in einem **gerichteten azyklischen Graph** (z.B. Baum), in dem jeweils - rekursiv - nach der Überprüfung eines Knotens zuerst dessen Töchter überprüft werden, bevor es mit seinen Geschwisterknoten weitergeht.
 - **Wegesuche**: finde das Suchelement in einem (meist: gerichteten azyklischen) **Graph** (kein Baum!) den besten Weg zum Suchelement.
 - * **A*Algorithmus**
 - * **Dijkstra's Algorithmus**
 - * weitere: → <https://en.wikipedia.org/wiki/Pathfinding>
- **Sortieren** meint, die richtigen Plätze für die Elemente einer unsortierten Liste zu finden, sie geschickte dort zu platzieren (und die sortierte Menge als Erfolgskriterium zurückzugeben):
 - **Insertion Sort**: → https://en.wikipedia.org/wiki/Insertion_sort
 - **Selection Sort**: → https://en.wikipedia.org/wiki/Selection_sort
 - **Bubble Sort**: → https://en.wikipedia.org/wiki/Bubble_sort
 - **Merge Sort**: → https://en.wikipedia.org/wiki/Merge_sort
 - **Quick Sort**: → <https://en.wikipedia.org/wiki/Quicksort>
 - **Shell Sort**: → <https://en.wikipedia.org/wiki/Shellsort>
 - **Heap Sort**: → <https://en.wikipedia.org/wiki/Heapsort>
 - weitere: → https://en.wikipedia.org/wiki/Sorting_algorithm

Addons:

- *Insertion Sort* und *Selection Sort* brauchen die *lineare Suche*.
- Die Taxonomiegrafik ist selbst ein Baum:
 - Bäume bestehen aus 0 - n Knoten.

- Knoten sind über Kanten direkt mit untergeordneten Knoten verbunden.
- Untergeordnete Knoten sind Tochterknoten des übergeordneten Knotens.
- Übergeordnete Knoten sind Väterknoten.
- In einem Baum hat jeder Knoten
 - * 0 - n Tochterknoten,
 - * 0 - 1 Väterknoten.
- Der Knoten ohne einen Vaterknoten ist der Wurzelknoten.
- Die Knoten ohne Tochterknoten sind die Blattknoten.
- Jede Kante ist ein Pfad.
- Ist X (z.B. [A B]) ein Pfad (von Knoten A zu Knoten B) und C ein Tochterknoten von B, dann ist auch (hier [A B C]) ein Pfad

2. Algorithusbewertung

Es gibt 3 gängige Kriterien zum Vergleich der Such- bzw. Sortialgorithmen untereinander:

- **Geschwindigkeit** (= Laufzeitverhalten), bezogen auf die Anzahl der Elemente (vermerkt in OH-Notation), ausgewiesen in *BestCase*, *Durchschnitt* und *WorstCase*
- **Ressourcenbedarf**: wie viel Memory braucht der Algorithmus zum Ausgangsmaterial
- **Stabilität**: garantiert der Algorithmus, dass die ursprüngliche Reihenfolge erhalten bleibt, sofern die Sortiervorgaben nicht ein Umpositionieren erzwingt? Beispiel bei Sortierung nach Nachnamen:
 - stabil: die ursprüngliche Sortierung wird (wo möglich) beibehalten:
 - * ({Anton First}, {Dorothea Last}, {Bertram First}) =>
 - * ({Anton First}, {Bertram First}, {Dorothea Last})
 - instabil: es ist nicht garantiert, dass ursprüngliche Sortierung (wo möglich) beibehalten wird.
 - * ({Anton First}, {Dorothea Last}, {Bertram First}) =>
 - * ({Bertram First}, {Anton First}, {Dorothea Last})

3. Übliche Klassifizierung

Eine übliche Gegenüberstellung sähe etwa so aus:

Algorithmus	Zeit (best case)	Zeit (avg. case)	Zeit (worst case)	Platz-bedarf	stabil
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ja
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Nein
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ja
Quicksort	$O(n * \log n)$	$O(n * \log n)$	$O(n^2)$	$O(\log n)$	Nein
Mergesort	$O(n * \log n)$	$O(n * \log n)$	$O(n * \log n)$	$O(n)$	Ja
Heapsort	$O(n * \log n)$	$O(n * \log n)$	$O(n * \log n)$	$O(1)$	Nein

nach

- → <https://www.happycoders.eu/de/algorithmen/sortieralgorithmen/>
- → <https://studyflix.de/informatik/sortieralgorithmen-1337>
- → <https://de.wikipedia.org/wiki/Sortiervverfahren>

Anmerkung:

Die 'Oh'-Notation besagt, dass bei jedem Fall von allen je konkreten Besonderheiten wie Rechnergeschwindigkeit, Speicherplatz oder Implementierung abstrahiert wird. So bleiben die Aussagen übrig, der Aufwand sei

- konstant: $O(1)$
- linear abhängig von der Anzahl der Elemente n
- logarithmisch abhängig von der Anzahl der Elemente $n * \log(n)$
- quadratisch / polynomial abhängig ist von der Anzahl der Elemente n^2
- exponentiell abhängig von der Anzahl der Elemente 2^n

Die Formeln zeigen:

```

1 'konstant = nicht abhängig von n' ist weniger als
2 'linear abhängig von n' ist weniger als
3 'logarithmisch abhängig von n' ist weniger als
4 'quadratisch abhängig von n' ist weniger als
5 'exponentiell abhängig von n'
```

vgl. dazu

- → <https://www.happycoders.eu/de/algorithmen/o-notation-zeitkomplexitaet/>
- → <https://de.wikipedia.org/wiki/Landau-Symbole>
- → https://en.wikipedia.org/wiki/Big_O_notation

4. Vorbereitung für Einzelanalyse

ÜBUNG LF11a:bj-01.sort-search:06

- ☐ Laden Sie sich aus Ihrem Downloadbereich den Ordner `bj-01.sort-search-snp.rose` herunter

Darin finden Sie Dateipaare nach dem Muster `rose-XYZ.py` und `rose-XYZ.sol.py`:

- Erstere enthält die Beschreibung der Aufgabe plus Hintergrundinformationen und Anregungen zur Implementierung.
 - Letztere enthält darüber hinaus eine ablauffähige eine Lösung in Python. (`python3 rose-XYZ.sol.py` oder Aufruf von VisualStudioCode aus)
-

5. Suchalgorithmen [→ ZP:Sheet:4]

(I) Lineare Suche

- A) notwendig, wenn innere Ordnung der zu sortierenden Elemente nicht bekannt
- B) meint, etwas sequentiell (linear) durchzugehen, bis Element dem Anfrageelement gleicht.
- C) Wann etwas 'gleicht',
 - a) hängt von der Domäne ab und
 - b) muss oft mit gesonderten Funktion 'equals' implementiert werden.
- https://de.wikipedia.org/wiki/Lineare_Suche
- https://en.wikipedia.org/wiki/Linear_search

(II) Binäre Suche

- A) setzt Sortierung der Suchdaten voraus
- B) erhält Suchdaten und einen Suchbereich
- C) hat der Suchbereich nur (noch) ein Element
 - a) gleicht es dem Suchelement, ist es gefunden
 - b) gleicht es dem Suchelement nicht, ist es nicht enthalten
- D) hat der Suchbereich mehrere Elemente

- a) berechnet sie daraus den mittigen Vergleichspunkt
- b) ist das Suchelement $<$ als der Vergleichspunkt,
 - * schränke Suchbereich auf alten Anfang bis Vergleichspunkt minus 1 ein
 - * starte binäre Suche dafür neu
- c) ist das Suchelement $\geq$ (als der) Vergleichspunkt,
 - * schränke Suchbereich auf Vergleichspunkt bis alter Endpunkt ein
 - * starte binäre Suche dafür neu

Auffallende Merkmale:

- Teilungsverfahren prozedural implementiert, nicht strukturell
- kann rekursiv oder iterativ implementiert werden
- https://de.wikipedia.org/wiki/Binäre_Suche
- https://en.wikipedia.org/wiki/Binary_search

(III) Binärer Suchbaum

- Suchdaten werden als Baum organisiert.
- Jeder Knoten hat höchstens 2 Töchter, die L-Tochter und die R-Tochter.
- Die Werte **aller direkten und indirekten L-Töchter** sind **kleiner als** der Wert des **Vaterknotens**.
- Die Werte **aller direkten und indirekten R-Töchter** sind **größer als** der Wert des **Vaterknotens**.
- Die Suchfunktion reduziert sich auf:
 - Hat der aktuelle Knoten keine Töchter:
 - * gleicht er dem Suchelement, ist es gefunden
 - * gleicht er dem Suchelement nicht, ist es nicht enthalten
 - Hat der aktuelle Knoten Töchter:
 - * ist der Wert des Suchelements kleiner als der Wert des aktuellen Knotens, mache mit seinem L-Knoten weiter
 - * ist der Wert des Suchelements größer als der Wert des aktuellen Knotens, mache mit seinem R-Knoten weiter

Auffallende Merkmale:

- Teilungsverfahren strukturell in den Datentyp integriert.
- Aufbau eines Binären Suchbaums programmiertechnisch komplex:
 - Hinzufügen eines Elements kann Reorganisation erfordern

- Um die volle Kraft für die Such zu entfalten, muss er ‘ausbalanciert’ (*balanced*) sein (~ gleich viele L- und R-Töchter).
 - Lohnt sich für Systeme mit signifikant weniger Datenerweiterungen als Datenzugriffe (z.B. in Dateisystemen)
 - https://de.wikipedia.org/wiki/BinÄrre_Suchbaum
 - https://en.wikipedia.org/wiki/Binary_search_tree
-

ÜBUNG LF11a:bj-01.sort-search:07

- Lösen Sie die Aufgaben
 - ☐ `sose-00-sequential-search.py`,
 - ☐ `sose-01-binary-search-iterative.py`
 - ☐ `sose-01-binary-search-recursive.py`
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in denen in den Dateien
 - ☐ `sose-00-sequential-search.sol.py`,
 - ☐ `sose-01-binary-search-iterative.sol.py` und
 - ☐ `sose-01-binary-search-recursive.sol.py`
-

6. Sortieralgorithmen:

6.1 Insertion Sort [→ ZP:Sheet:5]

Grundidee:

1. Nimm immer wieder das oberste Element vom Stapel der einzusortierenden.
 2. Gehe dann jedes Mal den Stapel der schon sortierten bis zu dem Element durch, das nach dem neu einzusortierenden kommen müsste.
- Füge das neu einzusortierende Element direkt davor ein.
 - <https://de.wikipedia.org/wiki/Insertionsort>
 - https://en.wikipedia.org/wiki/Insertion_sort
-

ÜBUNG LF11a:bj-01.sort-search:08

- ☐ Lösen Sie die Aufgabe [sose-02-insertion-sort.py](#),
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in [sose-02-insertion-sort.sol.py](#)
-

6.2 Selection Sort [→ ZP:Sheet:8]

Grundidee:

1. Gehe immer wieder den Stapel der noch einzusortierenden Elemente bis zu dem nächsten einzusortierenden durch (der direkte Nachfolger des zuletzt einsortierten).
2. Sortiere es nach dem zuletzt einsortierten ein.

- <https://de.wikipedia.org/wiki/Selectionsort>
 - https://simple.wikipedia.org/wiki/Selection_sort
 - https://en.wikipedia.org/wiki/Selection_sort
-

ÜBUNG LF11a:bj-01.sort-search:09

- ☐ Lösen Sie die Aufgabe [sose-03-selection-sort.py](#),
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in [sose-03-selection-sort.sol.py](#)
-

6.3 Bubble Sort [→ ZP:Sheet:9]

Grundidee:

1. Gehe den Stapel der zu sortierenden Elemente immer wieder von vorne durch
2. Tausche dabei immer wieder die benachbarten Karten miteinander, die in der falschen Reihenfolge stehen
3. Musstest Du dabei keine Karten mehr tauschen, bist Du fertig.
4. Sortiere es nach dem zuletzt einsortierten ein.

Die 'optische' Idee ist dabei,

- die 'richtigen' Karten steigen - wie Blasen - schrittweise von unten nach oben auf ihren Platz,

- umgekehrt sinken die ‘falschen’ von oben nach unten auf ihren Platz
 - → <https://de.wikipedia.org/wiki/Bubblesort>
 - → https://en.wikipedia.org/wiki/Bubble_sort
-

ÜBUNG LF11a:sbj-01.sort-search:10

- ☐ Lösen Sie die Aufgabe `sose-04-bubble-sort.py`,
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in `sose-04-bubble-sort.sol.py`
-

6.4 Quick Sort [→ ZP:Sheet:10]

Grundidee:

- A) Du bist ein intelligenter Sortieragent und kannst beliebig viele andere intelligente Sortieragenten mit Zuarbeiten beauftragen.
- B) Hat Deine Auftragsliste ≤ 1 Elemente, gib sie als sortierte Liste zurück.
- C) Ansonsten
 - a) suche Dir ein vom Wertebereich Deiner Auftragsliste möglichst mittig liegendes Vergleichselement.
 - b) sortiere Deine Auftragsliste in zwei Teillisten
 - * eine mit den Elementen, die kleiner sind als das Vergleichselement,
 - * die andere mit den Elementen, die $\geq$ als das Vergleichselement sind
 - c) Beauftrage zwei andere Agenten mit der Sortierung der Teillisten
 - d) Nimm deren Ergebnisse in Empfang
 - e) Hänge die sortierte Teilliste mit den im Vergleich zu Deinem Vergleichselement $\geq$ Elementen an die Teilliste mit den im Vergleich zu Deinem Vergleichselement $<$ Elementen

Anmerkung:

- Die Intelligenz der Agenten besteht darin, geschickt das ‘mittige’ Vergleichselement aus der Eingangsliste herauszusuchen.
- Das ‘mittige’ Vergleichselement heißt Pivotelement.

- Dessen Bestimmung muss geschickt implementiert werden.
 - Eine schlechtere Lösung wäre, dafür die Eingangsliste iterativ durchzugehen (lineare Suche).
 - Eine bessere Lösung wäre, in Referenzdaten die Mitte abfragen zu können.
 - Aber: Referenzdaten (vorab) aufzubauen, wäre für große Datenmengen zu zeit- und speicherplatzintensiv
 - Wenn beides nicht praktikabel ist: Merge-Sort
 - → <https://de.wikipedia.org/wiki/Quicksort>
 - → <https://en.wikipedia.org/wiki/Quicksort>
-

ÜBUNG LF11a:sbj-01.sort-search:11

- ☐ Lösen Sie die Aufgabe `sose-05-quick-sort.py`,
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in `sose-05-quick-sort.sol.py`
-

6.5 Merge Sort [→ ZP:Sheet:11]

Grundidee:

- A) Du bist ein intelligenter Sortieragent und kannst beliebig viele andere intelligente Sortieragenten mit Zuarbeiten beauftragen.
- B) Hat Deine Auftragsliste ≤ 1 Elemente, gib sie als sortierte Liste zurück.
- C) Ansonsten
 - a) Teile Deine unsortierte Auftragsliste in zwei kleinere (möglichst gleich große), immer noch unsortierte Teillisten auf.
 - b) Beauftrage zwei andere Agenten mit deren Sortierung.
 - c) Nimm deren Ergebnisse in Empfang.
 - d) Lege eine Ergebnisliste als Übernahmeliste fest, die andere als Vergleichsliste.
 - e) Hänge jetzt von vorn beginnend alle Elemente der Übernahmeliste an Deine Ergebnisliste an, solange diese kleiner sind als das erste Element Vergleichsliste und die Übernahmeliste nicht leer ist.

- f) Wechsle die Rollen der Liste und gehe zu e) zurück
- g) Hänge die Elemente der vollen Liste an Deine Ergebnisliste an
- f) Gib diese als Deine sortierte Liste zurück.

Unterschied zum Quicksort:

- Sortiarbeit auf dem “Rückweg”
 - Eignet sich auch, wenn Pivotelement nicht ermittelbar.
 - → <https://de.wikipedia.org/wiki/Mergesort>
 - → https://en.wikipedia.org/wiki/Merge_sort
-

ÜBUNG LF11a:bj-01.merge-search:12

- ☐ Lösen Sie die Aufgabe `sose-06-merge-sort.py`,
 - ☐ Vergleichen Sie Ihre Lösung danach mit der in `sose-06-merge-sort.sol.py`
-

6.6 Shell Sort [→ ZP:Sheet:12]

Grundidee:

- Der *Insertion Sort* hat im Idealfall eine Komplexität von n = hängt linear von der Elementzahl ab.
- Warum also diese unsortierte Menge nicht so *vorsortieren*, dass sie dem Idealfall möglichst nah kommt.
- Methode der Vorsortierung:
 - A) Gegeben sei eine ‘Abstandsfolge’ und eine Reihe zu sortierende Elemente.
 - B) Nimm die größte Zahl aus der Abstandsfolge, die gerade noch kleiner/gleich ist als die Hälfte der Anzahl der zu sortierenden Elemente.
 - C) Spalte die Reihe der zu sortierenden Elemente in entsprechend lange Subreihen.
 - D) Schreibe die Subreihen untereinander in eine Matrix
 - E) Sortiere die Spalten der Matrix.
 - F) Wenn die Länge der Reihen der Matrix > 2 :
 - * a) Nimm die größte Zahl aus der Abstandsfolge, die gerade noch kleiner/gleich ist als die Hälfte der Anzahl der Subreihen.

- * b) Spalte alle Reihen der Matrix in entsprechend lange Subreihen
- * c) Schreibe die Subreihen untereinander in eine neue Matrix
- * d) Sortiere die Spalten der Matrix.
- * e) GOTO (F)
- G) Wenn die Länge der Reihen der Matrix ≤ 2 , schreibe die Zahlen in eine neue Reihe zu sortierender Elemente.
- H) Sortiere diese Reihe mit dem *Insertion Sort*

Hinweise:

- erfunden wurde der *Shell Sort* 1959 von Donald L. Shell,
- hat also nichts mit Shellprogrammierung zu tun,
- Shell hat die Abstandsfolge $2, 4, 8, 16, \dots, 2^n$ vorgeschlagen
- hat **exponentielle Komplexität** $O(2^n)$
- *insertion sort* allein hat Komplexität $O(n^2)$ [also: kein Gewinn]
- deshalb viele andere Abstandsfolgen
 - $1, 3, 7, 15, 31, 63, \dots, 2^k - 1$ von Hibbard $O(n^{1.5})$
 - $1, 2, 3, 4, 6, 8, 9, 12, 16, \dots, 2^p \cdot 3^q$ von Pratt $O(n \cdot \log(n)^2)$
 - ...
- → <https://de.wikipedia.org/wiki/Shellsort>
- → <https://en.wikipedia.org/wiki/Shellsort>

6.7 Heap Sort Anwendung vom Binary Tree Sort